GUIA TÉCNICO

ETL e preparação de dados

Contratos, transformações, junções, rastreabilidade e validação

Propósito — conectar decisões metodológicas, exemplos reproduzíveis e interpretação responsável. Material complementar aos notebooks revisados, não substitui a execução no JupyterLab.

Como utilizar — Leia as explicações, execute os exemplos no ambiente indicado e consulte os notebooks correspondentes.

ROTEIRO DO DOCUMENTO

1. Desenhar o fluxo antes de transformar
2. Tipos, datas, ausências e anomalias
3. Deduplicação e granularidade
4. Junções e cardinalidade
5. Idempotência, escrita e leitura de retorno
6. Rastreabilidade e testes de invariantes

01 Desenhar o fluxo antes de transformar

ETL organiza extração, transformação e carregamento; em processos exploratórios, é útil separar ingestão, padronização, validação e publicação. Um fluxo reproduzível explicita a origem, o esquema esperado, o tratamento de exceções, o destino e as regras de auditoria. Os quatro CSVs da disciplina pertencem a `datasets/`; saídas calculadas ficam em `outputs/`. Essa separação reduz o risco de substituir o material de origem.

Ponto de atenção

A transformação não é confiável apenas porque termina sem erro: ela precisa preservar invariantes de negócio e produzir resultados explicáveis.

02 Tipos, datas, ausências e anomalias

Converta datas com política clara para valores inválidos; use números como números; valide quantidades, categorias e chaves. Imputar pela média muda a distribuição e depende do objetivo da análise. Em aprendizado de máquina, qualquer estatística de imputação ou escalonamento deve ser estimada apenas nos dados de treino para evitar vazamento; esse alerta é conceitual nesta disciplina de ETL.

Ponto de atenção

`errors='coerce'` converte erros em ausentes e exige inspeção posterior; `errors='raise'` interrompe o processo. A escolha é um contrato de qualidade, não preferência estética.

03 Deduplicação e granularidade

`drop_duplicates()` remove linhas completamente idênticas, mas não resolve duplicações de entidade nem confirma que duas linhas iguais são erros. Especifique uma chave candidata, uma política de precedência e um registro do número de exclusões. Se uma venda for registrada duas vezes por falha de importação, a soma duplicada aumenta o volume; mas vendas com campos iguais podem ser transações diferentes.

Ponto de atenção

Não elimine duplicatas antes de medir seu impacto e justificar a regra.

04 Junções e cardinalidade

Em um `merge`, é essencial saber se esperamos um-para-um, muitos-para-um ou muitos-para-muitos. Para acrescentar atributos regionais a vendas, `regiao` deve ser única na tabela de referência. `validate='many_to_one'` detecta multiplicações imprevistas; `indicator=True` expõe registros sem correspondência. Compare a quantidade de linhas e totais antes e depois da integração.

Ponto de atenção

Uma junção que produz 20% mais registros sem novos fatos de origem pode inflar somas silenciosamente.

05 Idempotência, escrita e leitura de retorno

Um processo idempotente com as mesmas entradas e configuração deveria produzir dados equivalentes. Ordene linhas quando necessário, padronize formatos e não inclua timestamps variáveis em colunas que fazem parte do conteúdo esperado. Grave em `outputs/`, releia o arquivo e verifique quantidades, esquema e somas de controle. Separe validações executadas antes da escrita e após a leitura.

Ponto de atenção

Resultados em disco são artefatos verificáveis; o simples sucesso de `to_csv()` não garante semântica preservada.

06 Rastreabilidade e testes de invariantes

Uma execução auditável registra tamanho da entrada, exclusões, ausências, transformações e totais de saída. Testes com `assert` verificam invariantes como não negatividade, integridade referencial, unicidade da referência e estabilidade de linhas. Em produção, substitua `assert` por validações explícitas quando sua desativação por otimização for possível. Para dados novos, falhar de maneira visível é melhor que corrigir silenciosamente.

Ponto de atenção

O objetivo da rastreabilidade é permitir reproduzir e explicar cada diferença entre dados brutos e resultado final.

Decisões avançadas e testes de robustez

Os tópicos seguintes aprofundam decisões que o código sozinho não explica. Use-os ao adaptar as demonstrações a bases diferentes.

Contrato de dados e falha segura

Um contrato útil inclui colunas obrigatórias, tipo esperado, intervalo permitido, regras de unicidade e restrições para chaves. A rotina deve emitir uma falha clara quando uma condição não for satisfeita, em vez de converter silenciosamente valores inválidos. Além disso, mantenha um pequeno relatório com número de registros rejeitados e motivo. Em exercícios locais, assert é suficiente para demonstração; em serviços, prefira exceções e verificações explícitas.

Prova de que a junção não alterou o universo

Antes de cada merge, teste a unicidade da dimensão. Depois, compare contagem de linhas, soma de unidades e percentual de chaves sem correspondência. Uma junção esquerda com indicador expõe regiões que não receberam atributos de referência. Se a chave for composta, especifique todas as colunas necessárias. Nunca trate resultado muitos-para-muitos como válido apenas porque o pandas permitiu a operação.

Política para dados ausentes

Dados não medidos, indisponíveis ou não aplicáveis exigem soluções distintas. A imputação pode servir a modelos preditivos, mas uma coluna imputada não deve substituir o dado original sem registro de transformação. Preserve um indicador de ausência quando isso for relevante e documente parâmetros utilizados. Em tarefas com treino e teste, ajuste transformadores no treino e aplique no teste para impedir vazamento.

Teste de repetibilidade e proveniência

Execute a transformação duas vezes a partir dos mesmos arquivos em uma pasta limpa; compare a ordenação, o esquema, as contagens e as somas de controle. Um hash dos arquivos de entrada ajuda a identificar mudanças de versão. O relatório de execução deve incluir nome do arquivo, regras aplicadas e destino. Reprodutibilidade não significa apenas que o código roda, mas que sabemos que dados ele processou.

Exemplo reproduzível

Execute na pasta da disciplina, com os quatro CSVs originais em datasets/. O código abaixo é uma demonstração complementar e não altera os arquivos de entrada.

```
from pathlib import Path
import pandas as pd
base = Path('datasets'); saida = Path('outputs'); saida.mkdir(exist_ok=True)
vendas = pd.read_csv(base / 'vendas_exemplo.csv')
regioes = pd.read_csv(base / 'regioes_exemplo.csv')
assert regioes['regiao'].is_unique
integrado = vendas.merge(regioes, on='regiao', how='left',
                         validate='many_to_one', indicator=True)
assert len(integrado) == len(vendas)
assert (integrado['_merge'] == 'both').all()
resumo = (integrado.groupby(['regiao', 'produto'], dropna=False)
          .agg(unidades=('unidades', 'sum')).reset_index()
          .sort_values(['regiao', 'produto']))
resumo.to_csv(saida / 'unidades_por_regiao_produto.csv', index=False)
assert len(pd.read_csv(saida / 'unidades_por_regiao_produto.csv')) == len(resumo)
```

Como interpretar o resultado

Confira se as medidas calculadas correspondem à pergunta, se há observações ausentes e se as somas ou taxas podem ser comparadas. Os valores resultantes pertencem exclusivamente às bases sintéticas de ensino.

Exercícios e resoluções comentadas

Exercício 1 — Qual erro sugere uma junção com mais linhas que a entrada?

Resolução comentada: Verifique primeiro duplicações na chave do lado direito e a cardinalidade declarada.

Exercício 2 — Por que não usar a média de toda a base para imputar antes do split?

Resolução comentada: Porque utiliza informações que deveriam ficar exclusivamente no conjunto de teste, caracterizando vazamento.

Exercício 3 — Por que reler a saída?

Resolução comentada: Para detectar diferenças de esquema, valores, codificação e quantidade causadas pela serialização.

Checklist antes de publicar uma conclusão

- Pergunta, período e unidade observacional declarados.
- Datasets originais preservados; saídas apenas em outputs/.

- Ausências, filtros, chaves e denominadores explicitados.
- Cálculos e gráficos vinculados a uma interpretação verificável.
- Limitações dos dados sintéticos e das inferências declaradas.

Referências para continuidade

Consulte a documentação oficial de Pandas, NumPy, Matplotlib, Seaborn, Plotly, Bokeh e Altair conforme a biblioteca usada em cada notebook. As versões instaladas devem ser verificadas no próprio ambiente; requirements.txt fornece requisitos mínimos, não um lockfile. Licenciamento e identidade Material didático original preparado para o acervo EP, com exemplos derivados dos dados de demonstração do pacote. Licenças de código e conteúdo são mantidas na raiz do repositório.